

智练云姿——基于 RK3588 的智能健身动作分析系统

研究报告

摘要

随着全民健身的普及，家庭、社区等轻量化健身场景日益增多，但用户面临动作缺乏指导、训练效果难量化等痛点。传统手机端识别方案易导致设备发热与卡顿，而云端方案则存在隐私泄露风险与网络延迟。

针对上述痛点，本项目提出并构建了一套基于 RK3588 边缘计算平台的智能健身动作分析系统。系统围绕“感知—识别—分析—反馈”闭环设计，集成了高清摄像头、端侧 AI 推理、PyQt 可视化界面与移动端 APP，在设备本地即可完成全流程交互。

算法层采用 YOLOv8 进行运动模式分类，结合 YOLOPose 提取人体关键点，通过计算关节角度与轴线偏移量，实现对深蹲、俯卧撑等基础动作的计数、评分与纠错。为突破端侧性能瓶颈，项目进行了深度优化：使用 ISP 提升成像质量，RGA 实现高频图像变换，NPU 加速模型推理，并引入 EMA（指数移动平均）算法有效抑制了关键点抖动与短时遮挡干扰。

经测试，该系统在低延迟下实现了高精度反馈，运动模式识别准确率达 90% 以上，姿态关键点误差控制在 5 像素以内。系统在复杂环境鲁棒性与长时运行稳定性上表现优异，成功打造了一套可展示、重隐私、易落地的端侧智能健身解决方案。

关键词： 边缘计算；端侧 AI；人体姿态估计；动作识别与纠错；智能健身

第一部分 作品概述

1.1 功能与特性

本项目构建了一套智能健身动作分析系统，实现“感知—识别—分析—反馈”的本地化闭环交互。系统核心功能包括：

智能运动模式识别：系统能够自动精准判别深蹲、俯卧撑、仰卧起坐三种基础健身动作，无需用户手动切换模式，实现了无感化的运动类型智能分类。

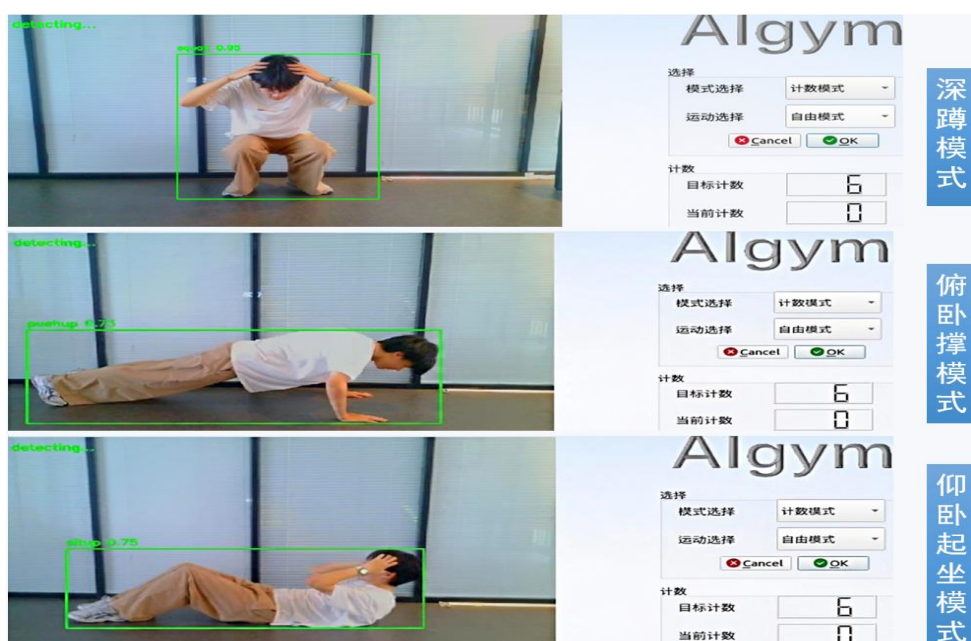


图 1 运动模式识别图

高精度姿态分析：基于 YOLOpose 算法，动态、实时提取人体 33 个骨骼关键点，并通过三维几何计算，秒级输出精确的关节角度与躯干轴线偏移量。



图 2 姿态识别图

即时纠正与量化反馈：结合多帧平滑状态机，用户可根据不同需求选择计数或计时模式；使用机器学习模型，系统可在运动过程中实时打分。



图 3 即时纠正与量化反馈

多终端运动数据管理：联动 PyQt 本地看板、Web 端与移动端 APP，自动记录训练次数、时长与热量消耗，生成日、周、月多维健康趋势图表，提供科学的运动规划支撑。

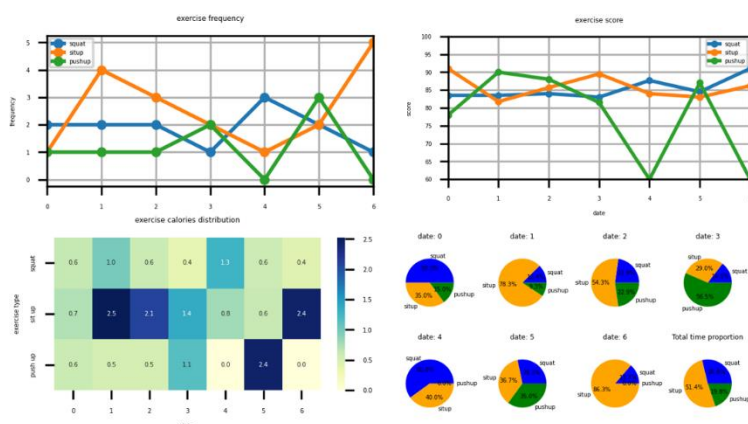


图 4 图表生成

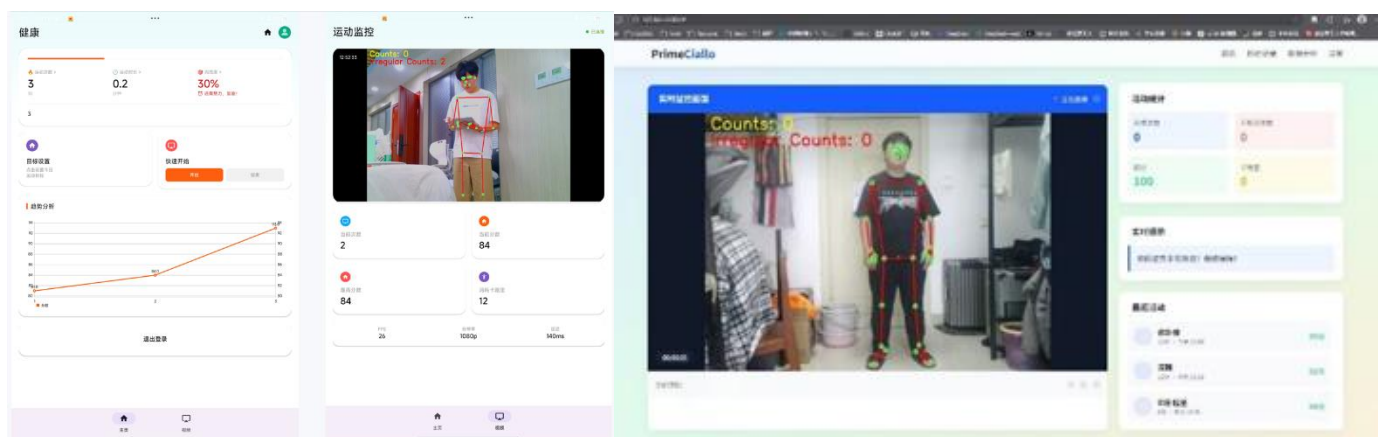


图 5 多终端运动数据管理

1.2 应用领域

本系统主要面向以下轻量化和本地化健身场景：

纯本地家庭健身场景：解决居家训练缺乏专业私教指导的痛点。系统完全在端侧本地运行，无需上传视频流，在提供实时动作纠偏与量化反馈的同时，彻底杜绝了家庭环境下的隐私泄露风险，且无高昂维护成本，极大地降低了大众科学健身的门槛。

智慧体育教学与校园体测：在传统健身房或学校体育课中，教练与教师难以同时精准兼顾多名学员。本系统可作为高效的辅助教学工具，持续记录动作次数、质量评分和错误类型。在校园体测中，可用于俯卧撑、仰卧起坐等项目的自动识别与辅助评分，减少人工统计误差并提升管理效率。

安全康复训练监测：康复训练极怕动作幅度过大或姿势错误造成二次损伤。系统能够精准监测关键关节角度与躯干动作幅度，确保患者在安全的医学范围内完成训练，并及时给予风险提醒，具备极强的社会推广与产业应用价值

1.3 主要技术特点

高性能纯本地边缘计算架构：系统打破传统方案对高带宽、低延迟网络的强依赖，将复杂的深度学习算法模型完全部署于纯本地端侧平台。从根本上杜绝了

因数据上传云端带来的家庭隐私泄露风险，并实现了毫秒级的超低延迟即时交互响应。

多源异构硬件流水线并行加速：首创了前端多硬件协同流水线，科学划分计算负载。由板载 ISP 完成动态成像画质优化，利用专用 RGA 2D 图形引擎进行高频图像尺度变换与色彩空间转换，释放 CPU 的密集搬运负载；核心推理完全由高性能专用 NPU 承担，CPU 仅负责轻量级状态机逻辑判定，充分发挥了 rk3588 的硬件性能，显著降低了设备长时运行的功耗与发热。

高鲁棒性运动状态联合判定机制：算法层创新性地融合了指数移动平均（EMA）平滑函数、双阈值阶段切换区间与连续多帧状态校验。该机制能够有效平毅人体关键点的局部噪点和轻微抖动，对健身过程中的短时遮挡或个别关键点丢失具备极强的自愈与抗干扰能力，大幅提升了动作计数的工程可靠性。

1.4 主要性能指标

本系统针对端侧智能健身动作识别与高频交互需求，在 1080P 视频流输入条件下进行了严格的软硬件协同测试，系统核心工程性能指标如下表所示：

评估维度	核心考核指标	实际测试表现
算法准确率	运动模式识别准确率	90% 以上
检测精度	姿态关键点定位误差	5 像素以内
实时响应	整体视觉推理延迟	100 ms 以内
画面流畅度	视频实时运行帧率	25~30 FPS
运行稳定性	资源开销与长时状态	CPU 占用约 20%，内存无泄漏

1.5 主要创新点

- (1) **纯本地端侧 AI 架构：**核心算法完全部署于边缘端，无需云端服务器协同，具备超低延迟交互与天然的隐私保护优势。
- (2) **双模型协同推理机制：**融合 YOLOv8 运动分类与 YOLOPose 关键点提取，将动作识别与深度质量评估有机结合，判定更精准。
- (3) **异构计算流水线加速：**科学划分 ISP、RGA、NPU 与 CPU 的计算任务，实现图像采集到推理控制的全硬件级加速，充分利用 rk3588 硬件性能。
- (4) **高鲁棒性联合判定算法：**引入 EMA 平滑与双阈值校验，有效抑制关键点抖动与短时遮挡，显著提升复杂场景下的工程可靠性。

1.6 设计流程

系统整体围绕“感知—识别—分析—反馈”的核心设计闭环展开。首先，利用高清摄像头进行视频流高频采集，通过底层的板端 ISP 优化成像画质，并交由专用 RGA 引擎进行硬件级像素缩放与色彩空间转换；随后，预处理后的图像输入 NPU 加速核心，并行调度 YOLOv8 与 YOLOPose 算法进行人体目标识别与关键点姿态特征提取；最后，结合 EMA 平滑的状态机对关节角度进行定量分析，计算结果实时输出至 PyQt 桌面端及移动端 APP 进行可视化交互反馈。

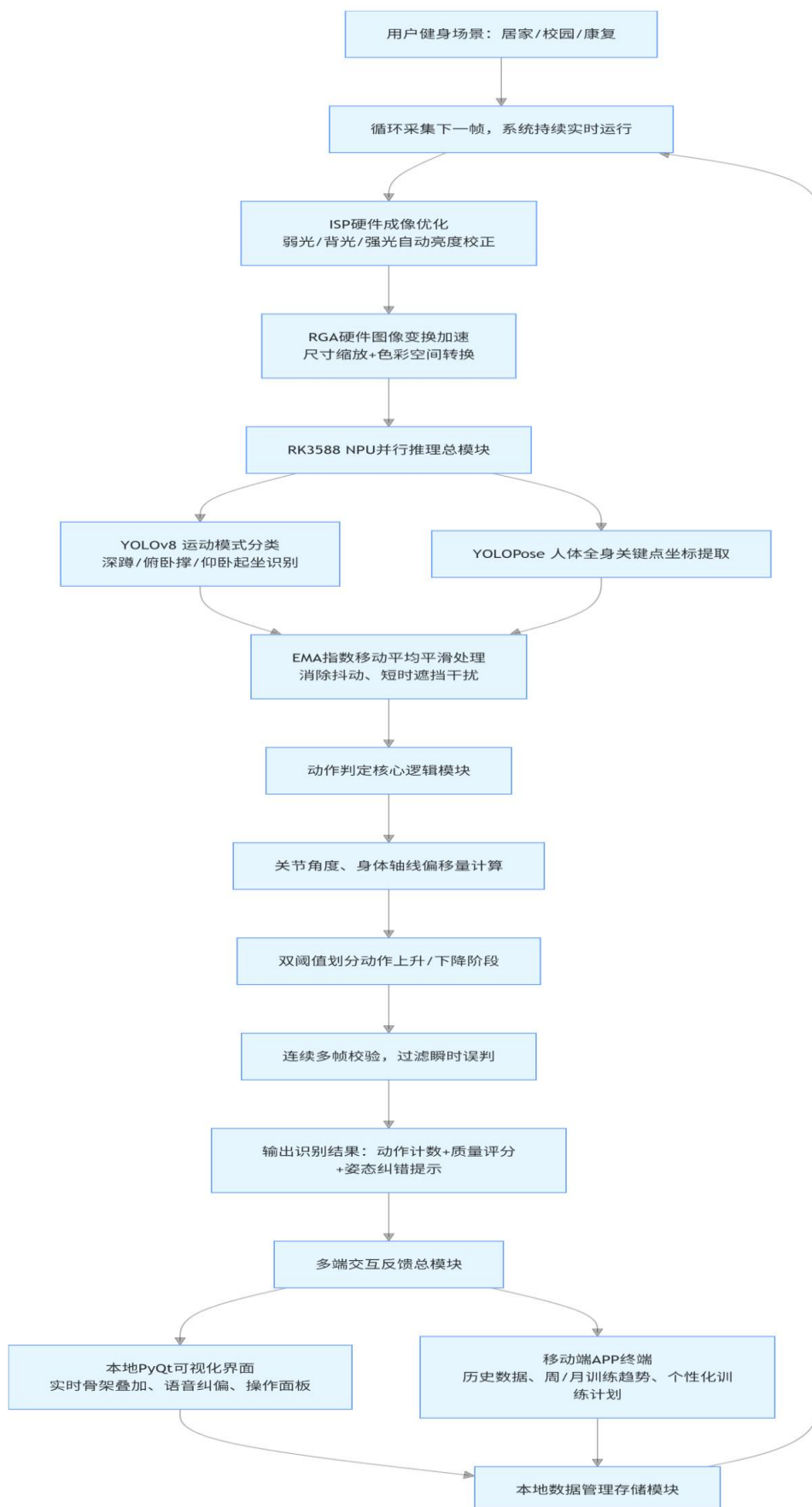


图6 设计流程图

第二部分 系统组成及功能说明

2.1 整体介绍

系统整体采用分层软件与异构硬件紧密结合的嵌入式软硬件架构，主要由图像采集与硬件预处理层、端侧边缘算法推理层、业务逻辑与判定状态机层，以及多端交互与数据反馈层四大核心模块组成。

各模块之间的协同与协同交互关系如下：首先，通过高清摄像头实时捕获运动画面的原生视频流，核心板 RK3588 的硬件 ISP 模块自动进行画质色彩优化，紧接着通过专用图形变换引擎 RGA 高速完成图像的尺寸裁剪与色彩空间转换，这一过程完全不占用 CPU 资源。随后，预处理完成的标准化视频帧直接送入 NPU 加速芯片，并行调度 YOLOv8 运动模式分类模型与 YOLOPose 人体姿态估计模型，高效提取出人体 33 个关键点的二维像素坐标。

提取到的高频关键点坐标流随即送入由 CPU 维护的动作判定状态机中。状态机结合 EMA（指数移动平均）算法滤除画面抖动引起的像素噪声，并通过关节角度几何解算与多阶段双阈值判定机制，精准完成动作计数与标准度评分。最终，计算得到的计数数值、实时错误类型及评分结果，一方面通过本地高带宽总线实时渲染在 PyQt5 桌面端交互看板上，配合语音模块提供即时交互反馈；另一方面，通过网络通信协议将清洗后的轻量化训练数据流同步至本地 SQLite 数据库与移动端 APP 客户端，实现跨终端的动态运动数据图表管理。各模块环环相扣，在本地端侧构建起一个超低延迟、重隐私保护的“感知—识别—分析—反馈”闭环生态

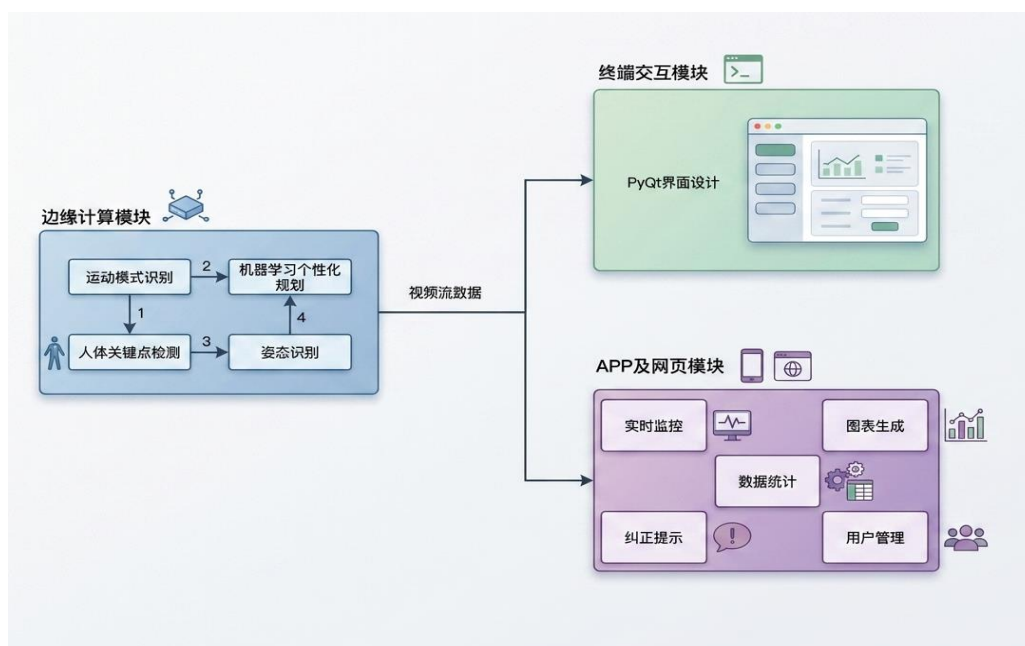


图 7 系统总体架构图

2.2 硬件系统介绍

2.2.1 硬件整体介绍；

系统硬件架构以国产高性能边缘计算芯片 RK3588 开发板为整套系统的核心控制与计算单元。该板卡集成了四核 Cortex-A76 和四核 Cortex-A55 架构的共 8 核 CPU、高性能 ARM Mali-G610 MP4 GPU，以及高达 6 TOPS 算力的专用 NPU 硬件加速核心。外围设备主要连接高清 MIPI/USB 摄像头作为底层的运动视频图像采集源；在输出端，系统通过板载 onboard 屏幕或外接 HDMI/eDP 显示器提供本地 PyQt 5 交互画面的流畅输出；同时，板卡集成了千兆以太网与双频 Wi-Fi 无线网络模块，用于向移动端智能手机 APP 客户端进行高频、轻量化的运动数据流同步。

2.2.2 电路各模块介绍

本设计电路主要涉及 ELF 2 以及 ASRPRO2.0 版本，ASRPRO2.0 版本原理图分别如图 8 所示：

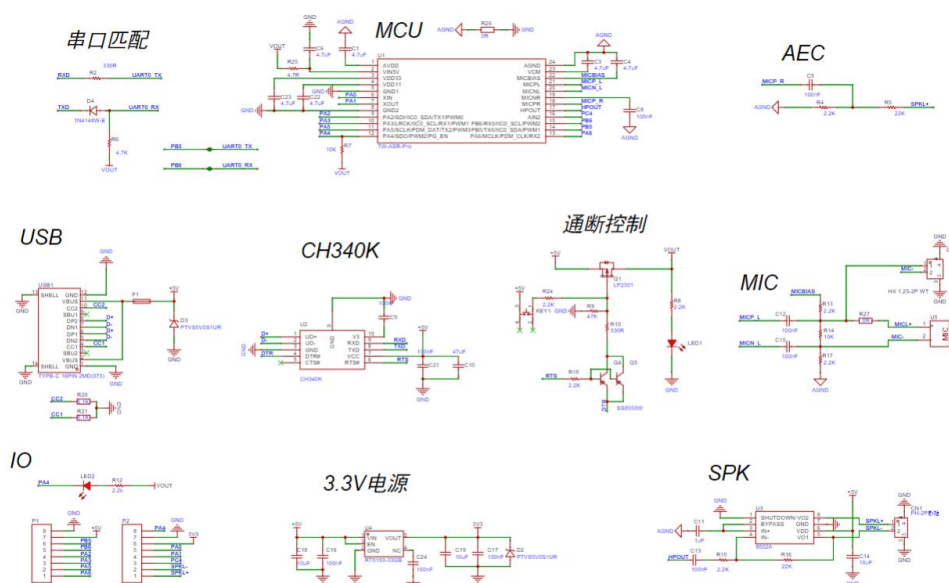


图 8 ASRPRO2.0 原理图

2.3 软件系统介绍

2.3.1 软件整体介绍

本系统软件架构运行在板载 Linux 操作系统之上，采用 Python 与 C++ 混合编程的闭环分层设计，主要由硬件加速驱动域、本地可视化交互域以及远程数据管理域组成。软件系统整体运行链路如下：由底层 V4L2 驱动高频捕获的高清视频帧，首先进入由 ISP 与 RGA 协同组成的硬件加速处理域，在像素级层面完成实时成像画质优化与标准尺寸格式变换，随后直接送入底层 RKNN-NPU 推理域，实现双模型的本地实时异构加速推理。计算得出的关键点坐标流送入核心业务逻辑域，由动作判定状态机动态输出计数与纠错结果。最终，这些高频交互结果一方面直接传递至基于 PyQt5 构建的本地可视化交互桌面，渲染出骨架叠加画面与进行语音实时播报；另一方面，通过网络通信协议将轻量化训练数据流同步至远程移动端 APP 客户端，完成跨终端的长期健康数据监控与趋势图表生成。

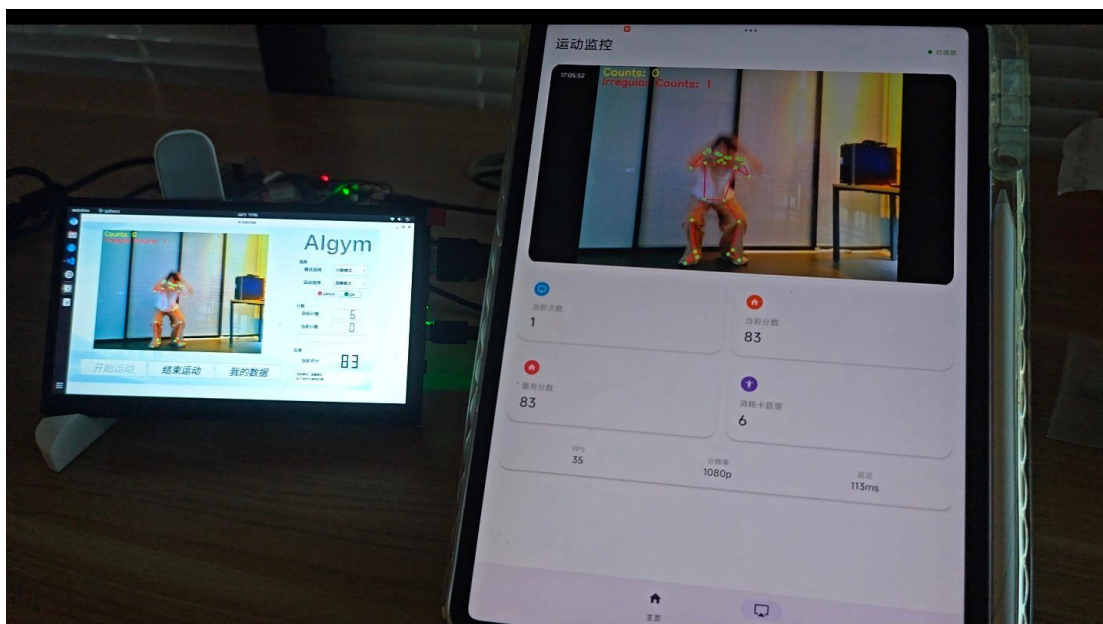


图 9 QT、APP 实时数据传输

2.3.2 软件各模块介绍

2.3.2.1 边缘计算模块

运动模式识别

运动模式检测模块作为系统的核心组件，主要负责从视频流中实时识别和跟踪人体运动，为后续的姿态分析提供基础数据。该模块基于 YOLOv8 模型^[1]实现，主要包含 YOLODetector 和 YOLOTracker 两个关键类，通过它们的协同工作完成从原始视频帧到结构化人体检测数据的转换。

当系统启动时，首先需要初始化 YOLODetector 类，该类负责加载预训练的 YOLOv8 模型权重文件，并配置检测所需的参数，如置信度阈值和交并比阈值。初始化完成后，模型就准备好对输入的视频帧进行处理了。

每帧视频进入系统后，YOLODetector 会对其进行前向推理，生成一系列检测框，每个检测框包含目标的位置、置信度和类别信息。由于实际场景中可能存在大量低置信度的检测结果和重叠的边界框，检测器会应用非极大值抑制算法，

过滤掉那些置信度低于阈值或与高置信度框重叠度过高的检测框，从而保留最可靠的检测结果。

接下来，YOLOTracker 类会接过处理流程，它的主要任务是对连续帧中的检测结果进行关联和跟踪，确保同一目标在不同帧中被识别为同一个对象。为了提高检测的可靠性，YOLOTracker 引入了连续帧确认机制，即只有当某个类别在连续多帧中都被检测到时，才会将其确认为有效检测结果。这种机制可以有效减少因偶然误检或遮挡导致的错误判断。

在跟踪过程中，YOLOTracker 会维护一个检测历史记录，记录每个可能的检测类别在最近几帧中的出现情况。通过分析这些历史数据，系统能够判断出哪些检测结果是稳定的，哪些是临时的噪声。一旦某个类别满足连续帧确认条件，系统就会输出该类别的确认信息，同时在视频帧上绘制相应的边界框和标签，直观地展示检测结果。

如果在连续多帧中都没有检测到任何有效目标，系统会进入等待状态，直到新的目标出现。这种设计使得运动模式检测模块能够高效地处理各种复杂场景，既保证了检测的准确性，又维持了系统的实时性能。

通过 YOLODetector 和 YOLOTracker 的紧密协作，运动模式检测模块能够准确、稳定地从视频流中提取人体目标信息，为后续的姿态识别和分类提供了坚实的基础，确保整个系统能够对用户的运动模式进行精确分析和判断。

该模块的工作流程如图 10 所示：

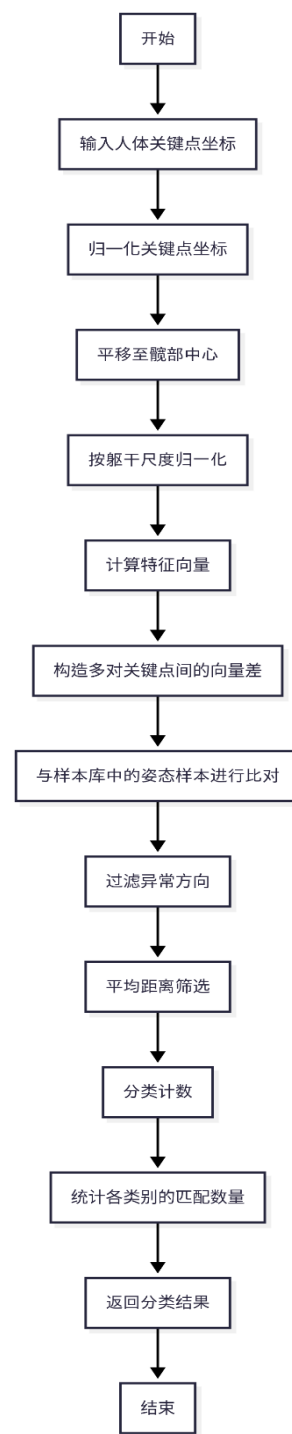
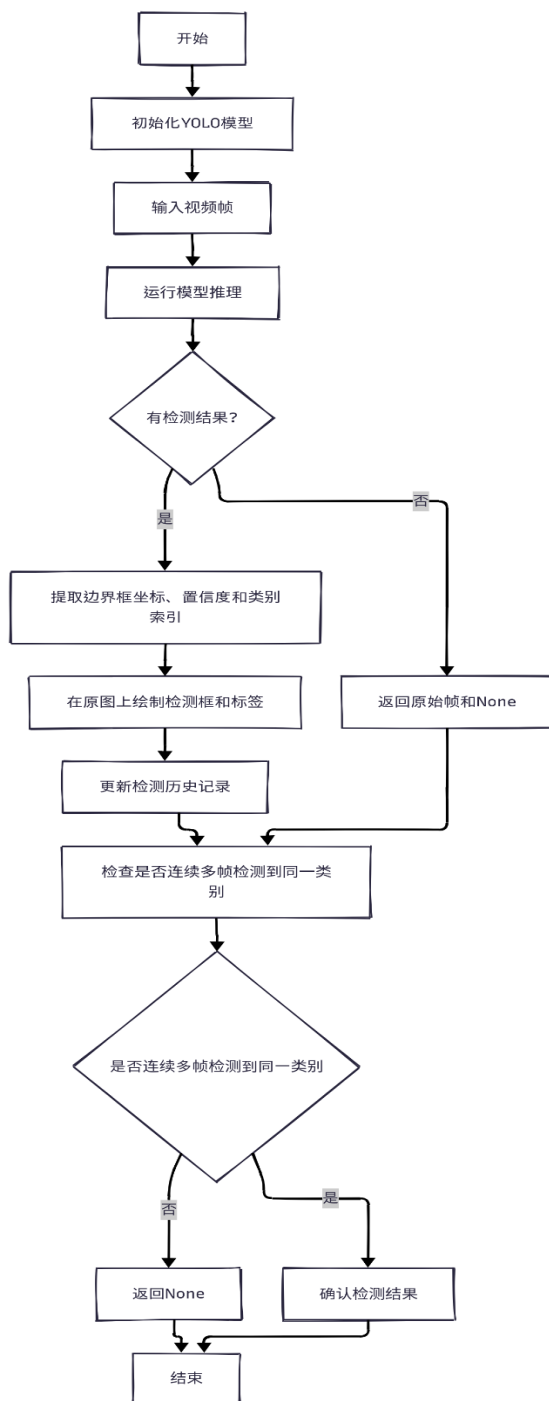


图 10 运动模式检测工作流程

图 11 姿态识别与分类

姿态识别与分类

姿态识别与分类模块的工作流程如图 11 所示，它是系统实现运动模式分析

的核心部分，主要通过 BodyPoseEmbedder 和 PoseClassifier 两个关键类协同工作，将原始的人体关键点数据转换为可识别的姿态类别。这一过程涉及从空间坐标到特征向量的转换，再到姿态分类的完整流程，为后续的动作计数和质量评估提供基础。

当系统通过 YOLO-Pose 模型^[2]获取到人体关键点坐标后，首先由 BodyPoseEmbedder 类对这些原始坐标进行预处理和特征提取。该类的核心任务是将三维空间中的关键点坐标转换为具有代表性的特征向量，以便后续进行姿态匹配。预处理过程包括两个关键步骤：首先，将所有关键点坐标相对于髋部中心进行平移，使坐标系原点位于人体的中心位置；然后，根据躯干的尺度对坐标进行归一化，消除因拍摄距离和人体大小差异带来的影响。

在完成预处理后，BodyPoseEmbedder 会计算一系列关键点对之间的向量差，这些向量差构成了姿态的特征表示。例如，通过计算肩关节与髋关节之间的向量、肘关节与肩关节之间的向量等，捕捉人体各部位的相对位置关系和角度信息。这种表示方式不仅能够描述人体的静态姿势，还能反映关节角度等动态信息，为姿态识别提供了丰富的特征。

接下来，PoseClassifier 类会利用这些特征向量进行姿态分类。该类维护了一个预先构建的姿态样本库，其中包含了各种标准姿态的特征向量及其对应的类别标签。当接收到新的特征向量时，PoseClassifier 会将其与样本库中的所有样本进行比对，计算相似度得分。

为了提高分类的准确性和鲁棒性，PoseClassifier 采用了多层次的筛选策略，类似于 KNN 聚类^[3]。首先，它会筛选出与输入特征向量方向最接近的前 N 个样本，通过最大距离度量排除那些方向差异较大的样本；然后，在这些候选样本中，进一步计算平均距离，选择平均距离最小的类别作为初步分类结果。这种双重筛选机制能够有效减少因局部关键点误差导致的误分类。

在获得初步分类结果后，系统会统计各类别的匹配数量，通过投票机制确定

最终的姿态类别。例如，如果在最近的几帧中，“俯卧撑下”姿态的匹配次数明显多于其他类别，则系统会确认当前姿态为“俯卧撑下”。这种基于时间连续性的判断方法能够有效平滑姿态切换过程中的波动，提高分类的稳定性。

通过 BodyPoseEmbedder 和 PoseClassifier 的协同工作，系统能够将抽象的人体关键点坐标转换为具体的姿态类别，为后续的运动计数和质量评估提供了明确的语义信息。这一过程不仅实现了从视觉数据到动作语义的跨越，还通过精心设计的特征提取和匹配算法，确保了姿态识别在复杂场景下的准确性和鲁棒性。

计数判断

计数判断模块是系统实现运动重复次数统计和动作质量评估的关键部分，主要通过 Counter 类实现。该模块接收来自姿态识别模块的实时分类结果，结合时间连续性和阈值判断机制，精确识别动作的起始和结束状态，从而完成计数并检测不规范动作。这一过程涉及状态机设计、阈值比较和时序分析等技术，确保计数的准确性和鲁棒性。

当系统启动时，首先需要初始化 Counter 类，配置计数所需的参数，包括目标动作类别名称、进入状态阈值、退出状态阈值以及动作的最小持续帧数。这些参数根据不同的运动类型进行调整，例如俯卧撑的“下”和“上”状态可能需要不同的阈值设置。初始化完成后，Counter 类开始监听姿态分类结果，准备进行计数判断。

每收到一帧的姿态分类结果，Counter 类会首先检查当前是否处于目标动作的执行状态。如果不处于执行状态，系统会比较当前姿态的置信度与进入状态阈值：若置信度超过阈值，则认为动作开始，系统进入目标状态，并重置持续帧数计数器；若未超过阈值，则继续等待。这种机制确保只有当姿态足够明确时才开始计数，减少误触发。

一旦进入目标状态，Counter 类会持续监控姿态置信度。当置信度低于退出状态阈值时，系统需要进一步判断动作是否完整执行。此时，持续帧数计数器发

挥作用：如果持续帧数小于预设的最小持续帧数，说明动作执行不完整，系统会记录一次不规范动作；如果持续帧数满足要求，则认为完成了一次有效动作，计数器加一。无论动作是否有效，系统都会退出目标状态，并重置持续帧数计数器，准备下一次动作的检测。

在目标状态持续期间，若姿态置信度始终保持在退出阈值以上，系统会不断增加持续帧数，确保能够准确捕捉动作的完整周期。这种基于时序分析的计数方法，有效避免了因姿态瞬间波动导致的误判，提高了计数的稳定性。

Counter 类还提供了实时反馈功能，通过累计的重复次数和不规范动作次数，用户可以直观地了解自己的运动情况。例如，在进行俯卧撑训练时，系统可以实时显示已完成的有效次数和不规范动作的次数，帮助用户调整动作姿势，提高训练效果。

通过这种状态机驱动的计数机制，Counter 类能够精确识别各种周期性运动的重复次数，并对动作质量进行有效评估。整个流程紧密结合姿态识别结果，通过合理设置阈值和时序分析，确保了计数判断在复杂运动场景下的准确性和可靠性，为用户提供了量化的运动数据支持。

计数判断模块的工作流程如图 12 所示。

运动计划决策

运动计划决策模块流程图如图 13 所示，它通过分析用户历史运动数据，运用随机森林算法学习^[4]运动模式与偏好，进而生成个性化次日运动计划。系统首先收集用户历史运动数据，包括运动日期、时长、消耗热量、运动得分等信息，接着对数据进行清洗和特征工程处理。处理过程中，系统会提取时间特征（如星期几、是否周末）、运动表现特征（如持续时间、消耗热量、运动得分）以及历史趋势特征，这些特征将作为模型训练的输入。

完成数据预处理后，系统会检查可用数据量是否充足。若数据充足，则将其划分为训练集和测试集，用于训练和评估随机森林回归模型；若数据不足，则采用默认计划策略。在模型训练阶段，系统会自动调整参数以优化预测准确性，并通过交叉验证评估模型性能。

当需要生成次日运动计划时，系统会获取当前用户运动状态数据，进行特征提取与标准化后，输入到已训练的模型中。模型会综合考虑各种因素，预测用户次日适合进行的各类运动次数。最后，系统根据预测结果生成个性化运动计划，为用户提供具体的运动建议。随着用户不断使用系统并产生新的运动数据，模型会持续学习和优化，使运动计划越来越符合用户的实际需求和运动能力。

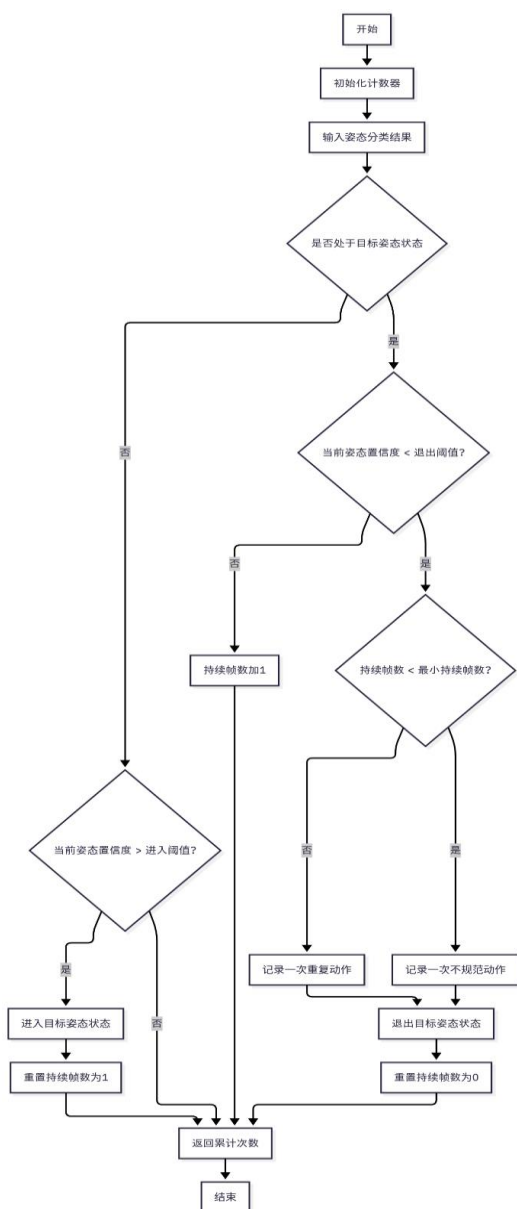


图 12 计数判断工作流程

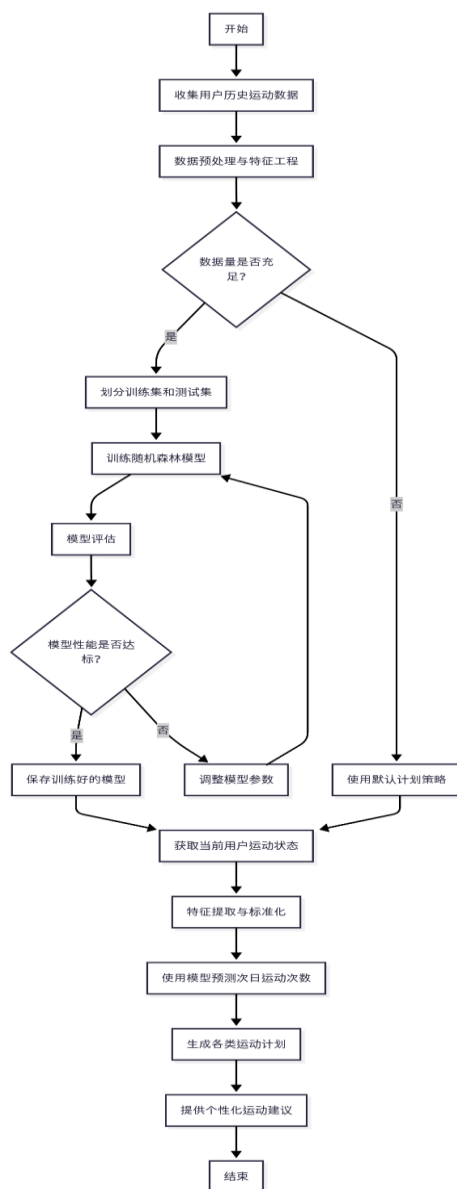


图 13 运动计划决策工作流程

2.3.2.2 终端交互模块

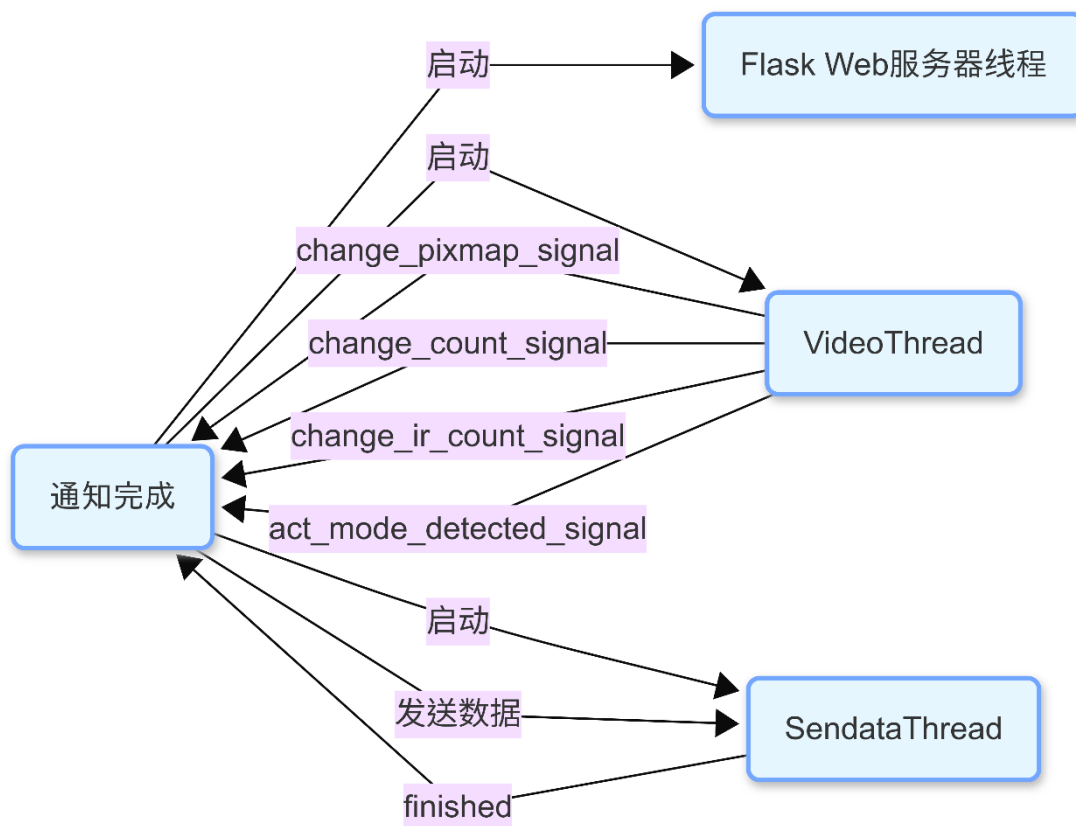


图 14 Qt 交互终端线程交互示意图

在本项目中，PyQt 主界面^[5]采用多线程管理模式，通过创建多个独立线程并行处理不同任务，避免了在主线程中执行耗时操作导致界面卡顿的问题。系统主要涉及三个核心线程：Flask Web 服务器线程、SendataThread 和 VideoThread，它们分别负责 HTTP 视频流服务、串口数据发送以及视频流处理。这种设计使程序能够同时处理视频分析、网络通信和设备控制等复杂任务，显著提升了系统的整体性能和响应速度。

各线程之间通过 PyQt 的信号与槽机制进行通信。VideoThread 在处理视频帧时，会实时发送 change_pixmap_signal 更新主界面的图像显示，发送 change_count_signal 和 change_ir_count_signal 更新计数信息，发送 act_mode_detected_signal 通知主界面活动模式的变化。主界面则可以通过调用

SendataThread 的方法来触发串口数据发送，当发送完成后，SendataThread 会发出 finished 信号通知主界面。这种基于信号的通信机制保证了线程间的高效协作，同时避免了直接共享资源带来的线程安全问题。

Flask Web 服务器线程

程序启动时，首先创建并启动一个专门的线程来运行 Flask Web 服务器^[6]。该服务器监听指定的 IP 地址和端口，提供两个核心路由：根路径/返回简单 HTML 页面用于浏览器测试视频流，/video_feed 则实时返回 MJPEG 视频流，供安卓 App 连接获取。Flask 服务器线程以守护线程模式运行，确保在主线程退出时能自动停止。这种设计使得系统能够在后台持续提供网络服务，而不影响主界面的操作。

SendataThread 串口通信线程

SendataThread 线程负责通过串口与外部设备通信，主要用于发送语音提示等数据。线程初始化时会创建串口连接，当主线程调用其 start()方法并传入待发送数据后，线程会将数据写入串口，等待短暂时间确保数据发送完成，随后自动停止。这种按需启动的模式有效避免了资源浪费，同时提供了一种可靠的方式与外部设备进行交互。

VideoThread 视频处理线程

VideoThread 是系统的核心处理线程，负责视频流的捕获、处理和分发。线程启动后，会根据配置决定是否首先进行模式检测：若启用模式检测，则通过 YOLO 目标检测器和追踪器处理视频帧，直至识别到目标；随后，线程会对每一帧中的人体姿态进行实时检测、分类和计数。处理后的视频帧会通过信号机制发送到主界面进行显示，同时也会传输到网络上供 Flask 服务器广播。这种设计使得视频处理与界面显示、网络传输能够并行进行，确保了系统的实时性。

通过这种多线程架构，系统实现了界面响应、视频处理、网络服务和设备控制的高效分离与并行处理。这种设计不仅提升了用户体验，确保界面操作流畅，

还使得系统能够充分利用多核处理器资源，应对复杂的实时处理需求。同时，模块化的线程设计也增强了代码的可维护性和可扩展性，为后续功能的添加提供了良好的基础。

2.3.2.3 APP 云端交互模块

用户信息管理

用户信息管理模块是应用程序的基础功能，用于管理用户身份验证和新用户注册，保障用户正常使用应用。该模块主要涉及 LoginActivity 和 RegisterActivity 两个活动，即登录和注册逻辑处理部分，以及对应的布局文件 activity_login.xml 和 activity_register.xml，即登录和注册界面。

当用户启动应用时，首先进入 LoginActivity。在 LoginActivity 的 onCreate 函数中，会进行一系列初始化操作，包括加载布局、获取 SharedPreferences 实例以检查用户是否已登录。若用户已登录，则直接跳转到主界面；若未登录，则显示登录页面，并添加淡入动画和按钮点击事件。用户在登录页面输入账号和密码后，点击登录按钮会触发 handleLogin 函数。此函数会检查输入是否为空，若为空则提示账号和密码不能为空；若不为空，则从 SharedPreferences 中获取保存的注册信息，验证输入的邮箱和密码是否匹配。若匹配，会保存登录状态，提示登录成功并跳转到主界面；若不匹配，则提示账号或密码错误。此外，若用户点击注册按钮，会跳转到 RegisterActivity，即注册界面。

在 RegisterActivity 中，onCreate 函数同样会进行初始化操作，如加载布局、获取 SharedPreferences 实例、添加淡入动画和注册按钮点击事件。用户在注册页面输入邮箱和密码后，点击注册按钮会触发 handleRegister 函数。该函数会检查输入是否为空，若为空则提示邮箱和密码不能为空；若不为空，则将邮箱和密码保存到 SharedPreferences 中，提示注册成功并结束当前页面，返回登录页。

为了实现数据的实时更新和连接状态的监控，模块还设置了定时器。在 ChartsFragment 和 VideosFragment 中，分别通过定时器定期更新图表数据、统计信息、时间戳和连接状态。同时，模块在多个地方添加了上下文检查，确保在 Fragment 仍然附加到上下文时才进行 UI 更新，避免出现空指针异常。在用户登出时，通过清除 SharedPreferences 中的登录状态，并跳转到登录页面，实现用户退出登录的功能。模块整体示意图如图 15 所示：

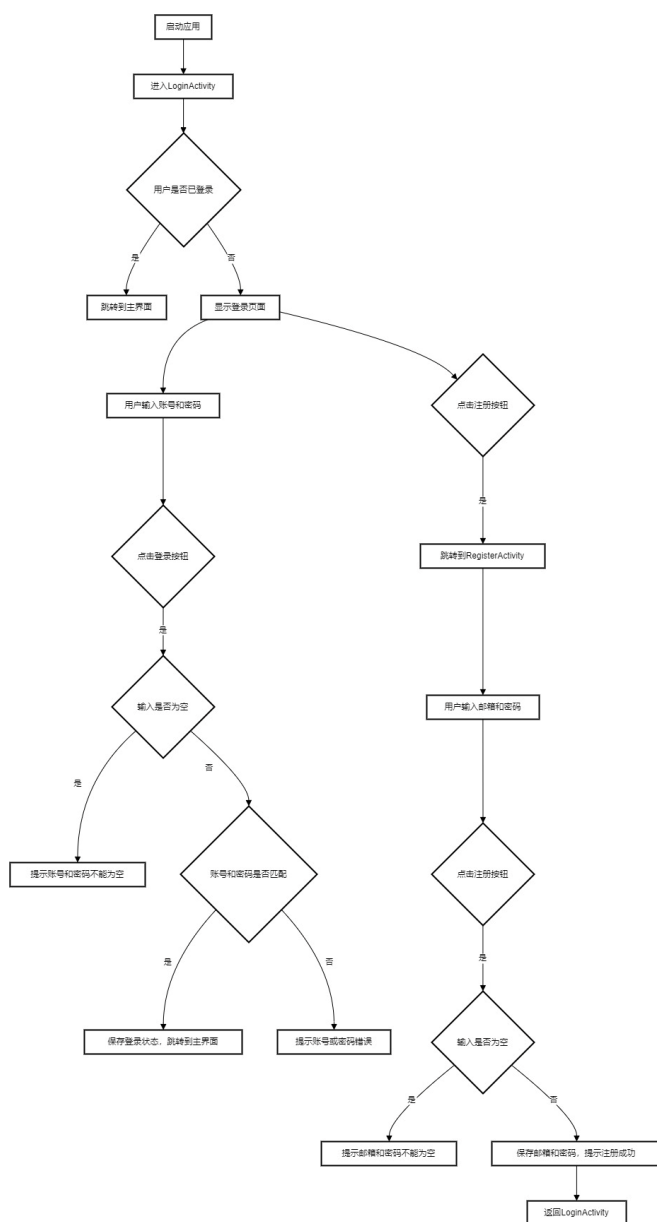


图 15 用户管理模块示意图

主页模块

主页模块是 Android 应用中为用户提供核心交互体验的重要部分，其主要功能包括展示个人信息、目标完成度以及运动数据等。当 Activity 启动后，会加载 HomeFragment。HomeFragment 首先调用 onCreateView 方法，使用 LayoutInflater 加载 fragment_home.xml 布局文件，并返回布局视图。接着进入 onViewCreated 方法，该方法会依次执行多个关键步骤。首先获取 SharedPreferences 实例，用于存储和读取用户相关数据。然后调用 initUIElements 方法，通过 findViewById 找到界面上的各个控件，设置目标按钮、目标完成度相关的文本视图和进度条、运动卡片相关的文本视图等，同时获取 ChartViewModel 实例，该实例用于获取图表数据。

随后调用 setupUserInfo 方法，从 SharedPreferences 中读取用户的昵称、个性签名和账号信息，并显示在界面上。同时，为昵称和个性签名的输入框添加文本变化监听器，当用户修改这些信息时，会将新值保存到 SharedPreferences 中。

setupClickListeners 方法为设置目标按钮和登出按钮设置点击事件监听器。当设置目标按钮被点击时，会调用 showGoalSettingDialog 方法，显示一个对话框让用户输入今日运动目标次数。用户输入后，程序会验证输入的有效性，若输入为大于 0 的有效数字，则将目标保存到 SharedPreferences 中，并更新目标完成度显示；若输入无效，则会给出相应的提示。当登出按钮被点击时，会清除 SharedPreferences 中的登录状态，并跳转到登录页面。

updateGoalProgress 方法会从 SharedPreferences 中获取今日目标，从 ChartViewModel 中获取当前完成次数，计算完成百分比，更新进度条和相关文本显示，并更新运动卡片数据。为了实现数据的实时更新，程序会调用 startDataUpdateTimer 方法，启动一个定时器，每 500 毫秒调用一次 updateGoalProgress 方法来更新目标完成度显示。

在布局方面，fragment_home.xml 定义了主页的界面结构。顶部是个人信息卡片，包含用户头像、昵称、账号、连续运动天数和个性签名，用户可直接在界面上修改昵称和个性签名。接着是目标完成度卡片，展示今日运动目标的完成情况，包括目标次数、当前完成次数和完成百分比，使用进度条直观显示。还有运

动卡片，实时显示运动次数和运动时长等数据。此外，还有快捷操作卡片，包含设置目标按钮，以及健康小贴士卡片和登出按钮。主页模块示意图如图 16 所示

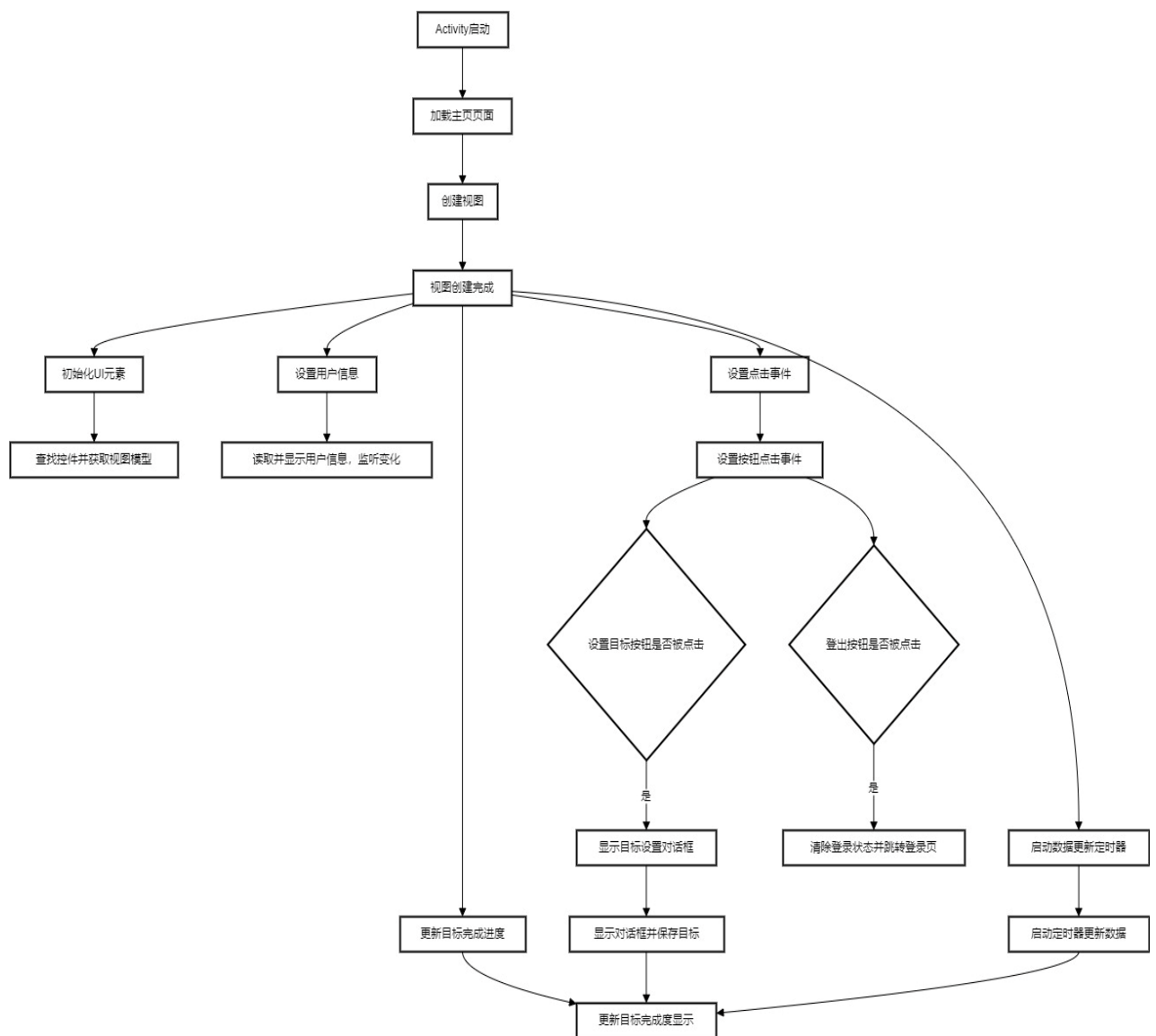


图 16 主页模块示意图

图表交互模块

图表模块是 Android 应用中负责展示运动分数变化趋势及统计信息的核心组件，主要由 ChartsFragment 和 ChartViewModel 构成。其工作流程始于应用启动，当 MainActivity 启动后，会先开启全局数据接收线程，该线程负责接收开发板 elf 传来的实时数据。一旦数据接收成功，便会立即更新到 ChartViewModel 中进行管理。

当用户导航至图表界面时，ChartsFragment 会被启动。在其初始化过程中，首先加载 fragment_charts.xml 布局文件，查找并绑定 LineChart 控件以及用于显示统计信息的 TextView 控件。接着获取 ChartViewModel 实例，开始初始化图表样式，包括设置图表描述、启用触摸交互、配置 X 轴和 Y 轴样式等。随后恢复之前的统计数据显示，并初始化连接状态指示器。最后启动一个数据更新定时器，用于定期刷新图表和统计信息。

数据更新定时器会按照预设的时间间隔触发更新操作。每次触发时，ChartsFragment 会从 ChartViewModel 获取最新的运动分数数据。如果数据不为空，将创建新的数据集并设置到图表中，同时更新可见 X 轴范围，确保用户能看到最新的数据点。之后会移动视图到最新数据位置，并更新统计信息显示，包括当前分数、最高分和平均分。如果数据为空，则重置统计信息显示。

在 ChartViewModel 类里面，当接收到新的运动分数数据时，会将其添加到内部的数据列表中，并更新统计信息。更新统计信息时，会比较新分数与当前最高分，若新分数更高则更新最高分，同时重新计算平均分。

在整个数据流动过程中，ChartsFragment 会持续检查与数据源的连接状态，并通过 UI 进行显示，确保用户了解数据接收情况。通过这种分层设计，图表模块实现了数据获取、处理和显示的分离，保证了系统的可维护性和扩展性，同时通过定时器机制确保了数据展示的实时性。图表交互模块示意图如图 17 所示：

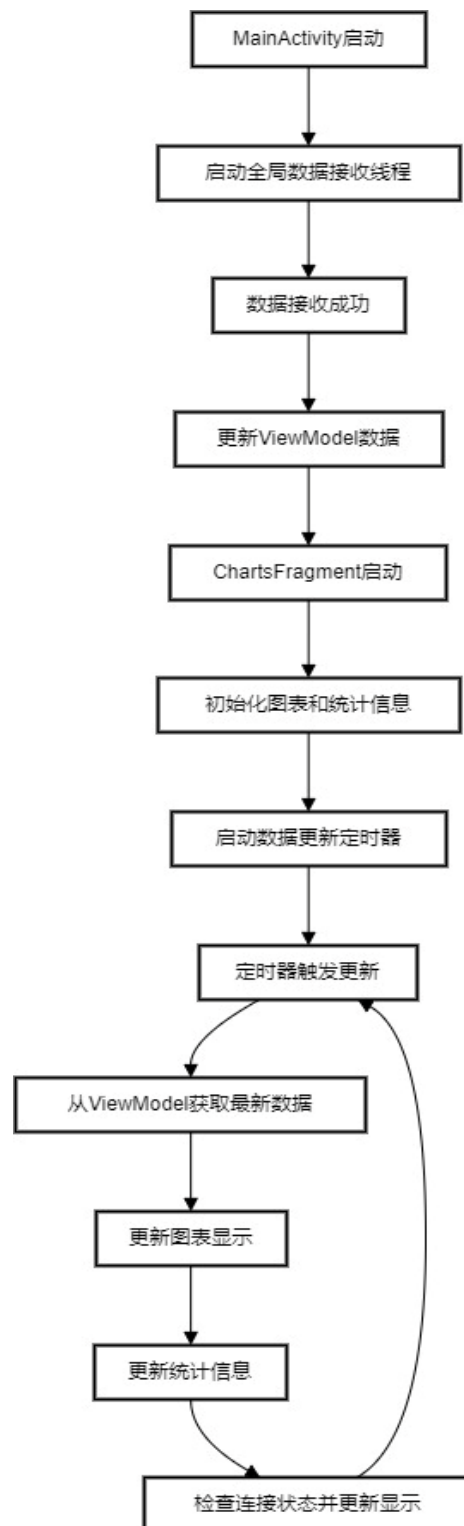


图 17 图表交互模块示意图

实时监控模块

视频实时监控模块的核心功能是从指定视频流地址获取实时视频数据[7]，在界面上进行显示，并提供视频统计信息，帧率、分辨率、延迟和连接状态的实时更新。首先，VideosFragment 通过 onCreateView 方法加载布局文件，创建视频监控界面的视图结构。随后，在 onViewCreated 方法中进行一系列初始化操作：初始化 UI 元素，包括状态指示器、时间戳显示、统计信息文本和运动分析相关控件；设置运动分析按钮的点击事件，用于触发运动分析功能；启动三个定时器分别处理时间戳更新、视频统计信息更新和实时数据更新。

时间戳更新定时器每秒调用 updateTimestamp 方法，获取当前时间并更新界面上的视频时间戳显示。视频统计信息更新定时器每 2 秒调用 updateVideoStats 方法，从视频流获取帧率、分辨率和延迟数据并更新界面显示。实时数据更新定时器每秒调用 updateRealTimeData 方法，从 ChartViewModel 获取当前分数、最高分数和运动次数等数据更新界面，并调用 updateWorkoutAnalysis 方法更新运动分析结果。

在整个过程中，updateConnectionStatusFromMain 方法会定期检查与开发板的连接状态，并更新界面上的连接状态指示器和文本显示。当用户点击运动分析按钮时，系统会显示提示信息，并根据当前运动数据生成运动建议、消耗热量等分析结果。所有这些操作都在确保 Fragment 已附加到 Activity 的前提下进行，以保证界面更新的安全性。实时监控模块示意图如图 18 所示：

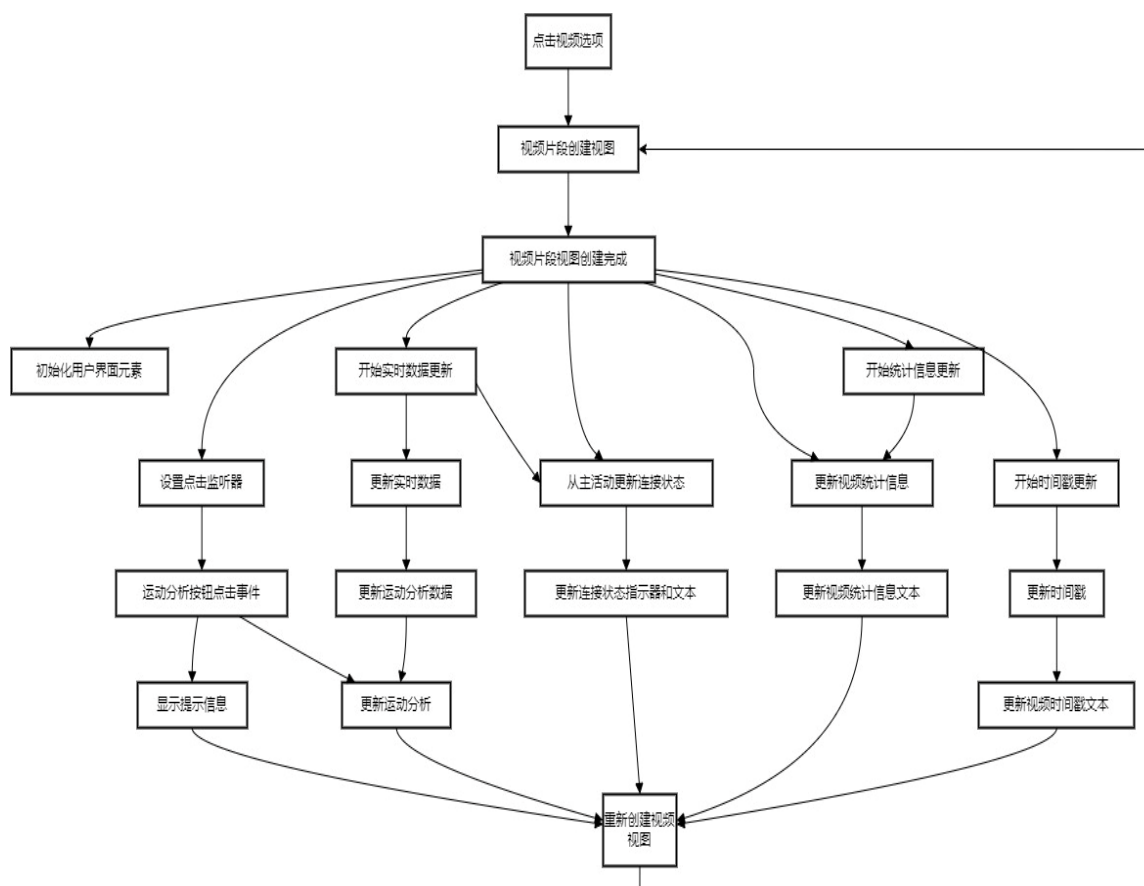


图 18 实时监控模块示意图

2.3.2.4 语音模块

该模块是一个基于嵌入式系统的语音播报控制程序，其整体流程从系统启动开始，首先执行 `setup()` 函数完成操作系统启动前的初始化，通过 `set_wakeup_forever()` 将系统设置为永久唤醒模式，不需要进入休眠，以保证该模块接受到开发板 `elf2` 传来正确数据能够及时处理，确保系统能持续运行并响应外部事件；随后进入 `hardware_init()` 函数进行操作系统启动后的硬件初始化，先通过 `vol_set(7)` 设置音量为 7，再通过 `setPinFun` 将引脚 13 和 14 配置为第二功能，即用于与开发板 `elf2` 进行串口通信，接着调用 `Serial.begin(9600)` 初始化串口通信，波特率设为 9600，之后通过 `xTaskCreate` 创建一个名为 `UART1_RX`、栈大小 256、优先级 4 的任务来处理串口接收数据。`UART1_RX` 任务启动后进入

无限循环，不断通过 Serial.available () 检查串口是否有数据，若有则通过 Serial.readString () 读取字符串数据并打印，接着判断接收的字符串前两个字节是否为 0x41(即字符 'A')，若是则进一步检查第三个字节：当第三个字节为 'A' 到 'I' 时调用 play_audio (10500) 播放对应语音，来提醒用户注意运动规范，每次处理后延时 2 毫秒继续循环监听，从而保证及时播报用户运动是否规范。语音模块示意图如图 19 所示

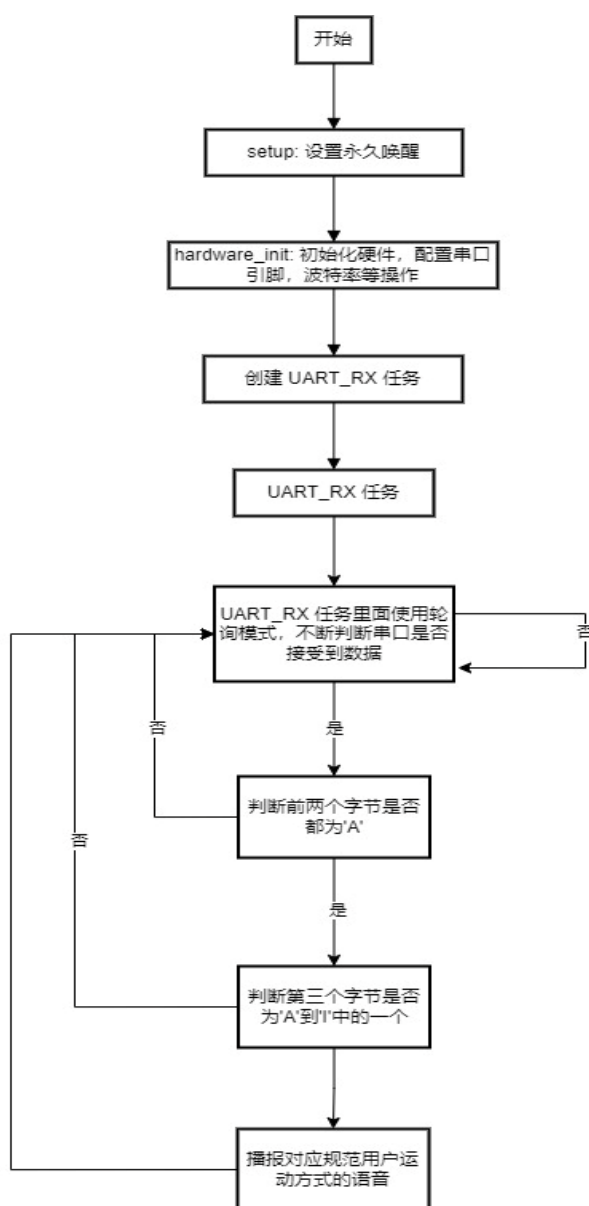


图 19 语音模块示意图

第三部分 完成情况及性能参数

3.1 整体介绍

目前，本端侧智能健身动作分析系统已完全实现了预期的全部工程化设计与算法功能，整机系统运行状态良好且表现稳定。系统成功将高性能嵌入式主板、高清视频采集前端、本地 PyQt5 可视化交互终端以及远端移动端管理 App 有机结合，进入了高可靠性的全闭环工作状态。在实际演示中，系统能够灵敏、自主地对深蹲、俯卧撑、仰卧起坐等多种基础健身动作进行全流程的端侧智能识别、骨架叠加、实时计数、动作评分与语音智能纠错反馈，达到了极高的完成度

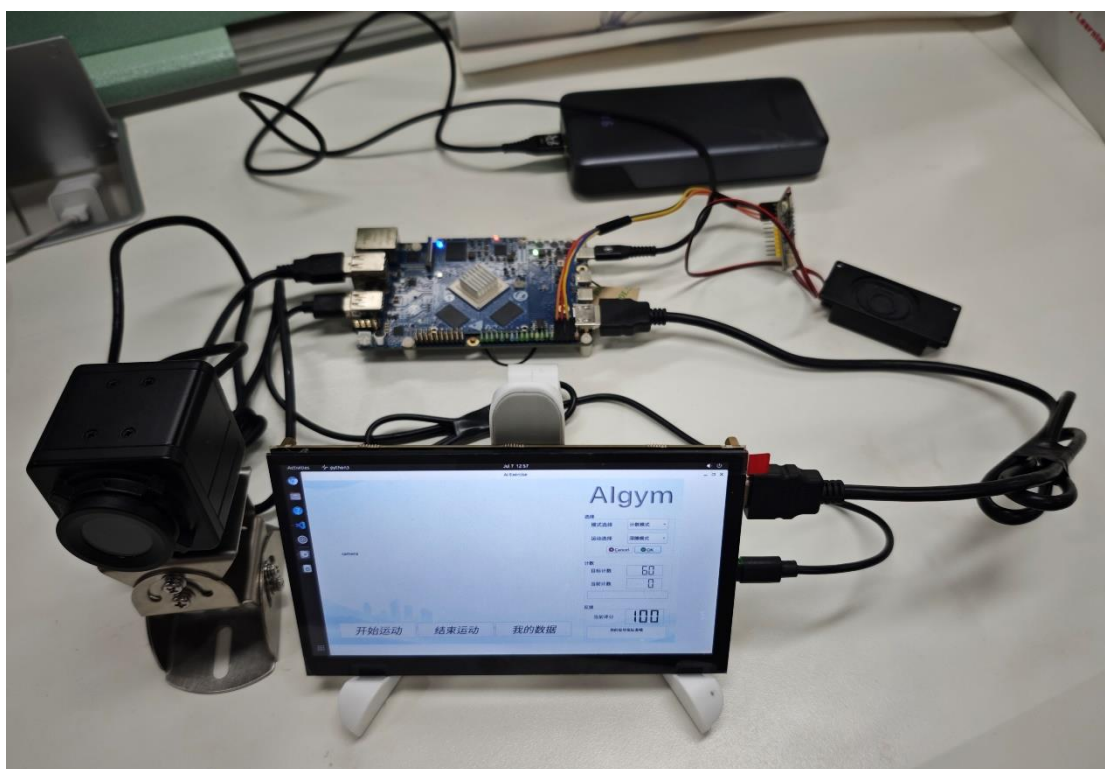


图 20 实物 45° 图



图 21 正面图

3.2 工程成果

3.2.1 电路成果

系统电路板卡焊接规范，元器件布局紧凑合理。12V DC 电源管理主电路及各级降压模块供电电流与电压表现十分平稳，未出现瞬态跌落；MIPI-CSI 摄像头高速信号接口与 HDMI 显示输出接口的差分走线信号完整性良好，整机连续运行中无任何由于高频信号干扰或阻抗不匹配导致的画面抖动与闪烁现象。



图 22 电路搭建图

3.2.2 软件成果

软件工程完成了 PyQt5 桌面端应用与移动端 APP 的全套界面开发与联调。本地 PyQt5 交互界面实现了高带宽视频帧的实时渲染，骨架关键点和骨骼连线动态叠加平滑无卡顿，实时计数看板与动作评分数值显示醒目；移动端 APP 成功实现了与本地 SQLite 数据库的数据同步，界面布局美观，能动态生成清晰的日、周、月健康趋势分析图表。

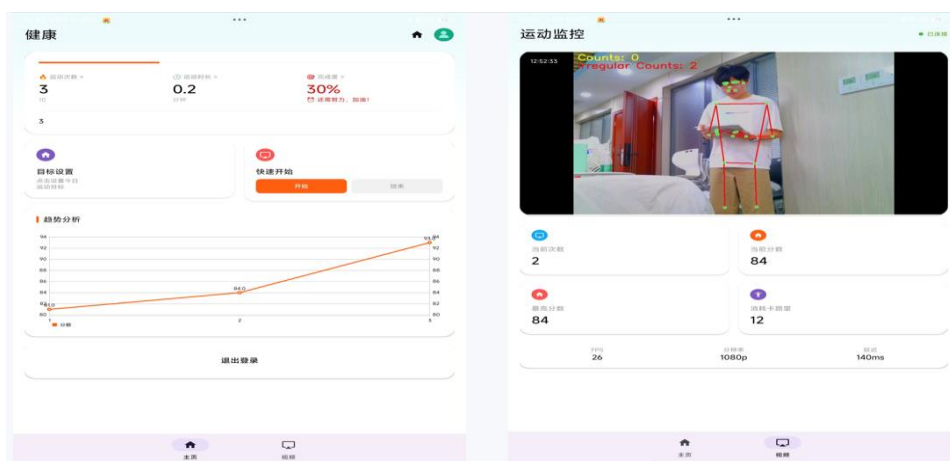


图 23 软件端展示

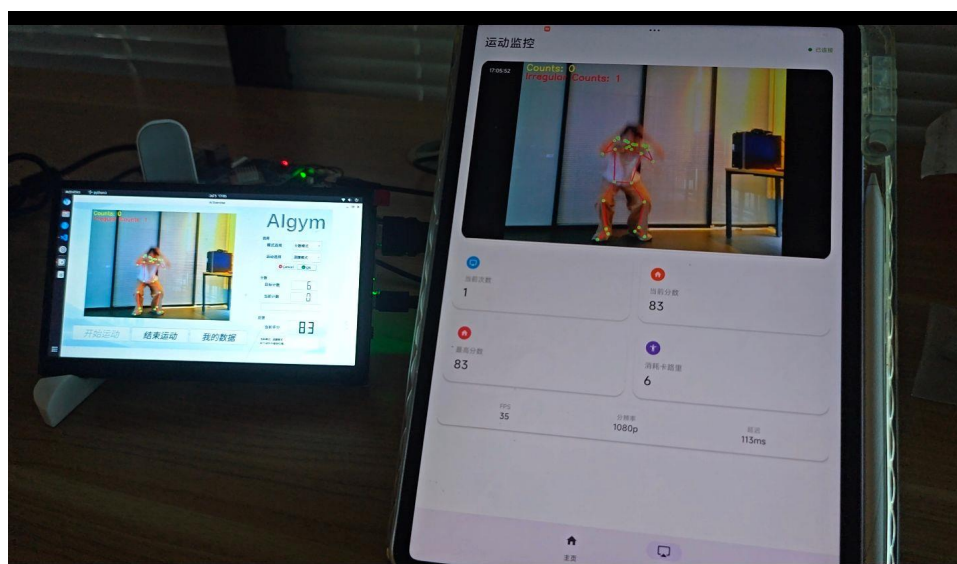


图 24 QT APP 实时交互



图 25 网页成果

3.3 特性成果

本系统围绕运动动作识别与智能计数应用需求，从图像预处理能力、视觉算法性能、系统鲁棒性以及系统稳定性四个维度构建综合评价体系。针对各项指标设计了相应的测试场景和验证方案，通过不同环境、不同目标对象以及长时间运行实验，对系统整体性能进行全面验证，从而确保系统具备较高的识别准确率、良好的实时性以及稳定可靠的运行能力。

3.3.1 图像预处理

图像预处理是人体姿态识别的重要前置环节，其处理效果直接影响后续关键点检测与动作识别的准确性。本系统主要验证在复杂光照环境下人体检测[8]与姿态估计的稳定性，包括正常光照、弱光环境以及背光环境等典型应用场景。

实验过程中分别采集不同光照条件下的人体图像，对比分析人体检测框、人体关键点定位及骨骼连线效果，验证图像增强算法及预处理流程对于视觉识别性能的改善能力。从实验结果可以看出，系统能够在不同光照条件下有效提取人体目标，保持人体轮廓完整、关键点定位准确，说明图像预处理模块能够有效降低环境光照变化带来的干扰，为后续动作识别提供稳定、高质量的输入数据。图 26 展示了系统在正常光照、弱光环境以及背光环境下的人体检测效果，可以看出系统均能够完成较为准确的人体定位和骨骼关键点识别，验证了图像预处理算法具有良好的环境适应能力。



图 26 复杂光照下的识别效果图

3.3.2 视觉算法性能指标

视觉算法性能主要考察系统在嵌入式平台上的实时推理能力、推理延迟以及整体处理效率。本项目围绕 RK3588 平台构建了完整的视觉推理链路，将视频采集、图像预处理、人体检测、姿态估计、动作判定以及结果显示等多个环节进行流水线优化，实现了低延迟、高实时性的运动识别系统。

系统首先由 ISP 完成图像采集、自动曝光、降噪及图像增强等处理，提高输入图像质量；随后利用 RGA 完成图像缩放、裁剪以及 RGB/YUV 色彩空间转换等预处理任务[9]，避免 CPU 参与大量像素运算，降低数据搬运开销。

在模型推理阶段，YOLOv8 与 YOLOPose 模型均部署至 RK3588 的 NPU 上运行，充分利用 NPU 对 INT8 神经网络计算的硬件加速能力，实现人体目标检测与关键点估计的高速推理。与此同时，CPU 仅负责运动状态机、计数逻辑、姿态评分及界面交互等轻量级任务，使推理计算与业务逻辑并行执行，提高系统整体吞吐能力。

此外，系统通过异构计算资源协同调度，将 ISP、RGA、NPU 和 CPU 各自负责最擅长的计算任务，形成“图像采集—图像预处理—模型推理—动作分析—结果显示”的流水线处理机制。相比全部由 CPU 完成处理的传统方案，大幅减少了 CPU 占用率，有效降低了系统整体推理延迟，提高了视频流处理效率。

实验结果表明，在 1080P 视频输入条件下，系统能够稳定完成实时人体姿态识别与动作计数，整体视觉推理延迟控制在 100 ms 以内，视频处理帧率保持在 25~30 FPS，满足实时运动识别应用需求。同时，在连续运行过程中，各模块能够保持稳定协同工作，无明显帧率波动或推理阻塞现象，验证了本系统视觉推理链路具有较高的实时性能和工程应用价值。

3.3.3 系统鲁棒性指标

系统鲁棒性主要用于评估系统在复杂实际应用环境中的适应能力和抗干扰能力。本项目分别设计了距离变化测试、人体姿态变化测试、多目标干扰测试以及背景复杂度测试等多种实验场景，对系统识别性能进行综合验证。

实验中，测试人员分别在近距离、中距离以及远距离范围内进行动作识别，系统均能够稳定检测人体目标并完成动作计数。同时，通过模拟人体部分遮挡、侧身站立、不同角度运动等复杂姿态变化，验证系统对于人体姿态变化的适应能力。此外，在背景中存在桌椅、门窗、健身器材等复杂物体，甚至多人同时出现在画面中的情况下，系统依然能够准确锁定目标人体，并保持较高的识别稳定性，有效避免背景干扰导致的误识别现象。

如图 27 所示，复杂障碍物遮挡下系统能够准确识别人体全身姿态；图 28 验证了系统在不同识别距离下仍具有较高的检测精度；图 29 展示了人体大角度运动过程中系统依旧能够保持稳定的人体关键点跟踪能力，充分证明系统具有较强的环境适应能力和鲁棒性。



图 27 复杂障碍物遮挡下人体识别



图 28 距离变化时人体识别



图 29 大角度复杂动作人体识别

3.3.4 系统稳定性指标

系统稳定性主要考察系统在长时间连续运行情况下的资源占用情况及运行可靠性，是评价工程化应用能力的重要指标。本项目通过持续运行测试，对 CPU 占用率、内存占用率、NPU 资源利用率、系统温度以及视频帧率等关键性能参数进行实时监测和统计分析。

在系统优化过程中，通过合理进行任务调度，将图像采集、视频解码、人体姿态推理、动作识别及界面显示等功能模块进行并行处理，充分发挥 ISP、GPU、CPU 和 NPU 等硬件资源协同工作的优势，有效降低 CPU 计算负载，提高整体运行效率。

实验结果表明，在系统连续运行的情况下，CPU 占用率始终保持在较低水

平，内存占用稳定，无明显内存泄漏现象；视频处理帧率保持稳定，系统响应及时，无明显卡顿、掉帧及程序异常退出等情况。资源监控数据显示，各硬件模块运行状态平稳，温度控制正常，系统能够持续保持良好的运行性能。

图 30 展示了系统运行过程中 CPU、内存等资源监控情况。从监测结果可以看出，系统整体资源利用率较为合理，各项性能指标均保持稳定，说明本系统具备较高的工程可靠性和长期运行能力，可满足家庭健身、校园体育教学及康复训练等实际应用场景对系统稳定性的要求。

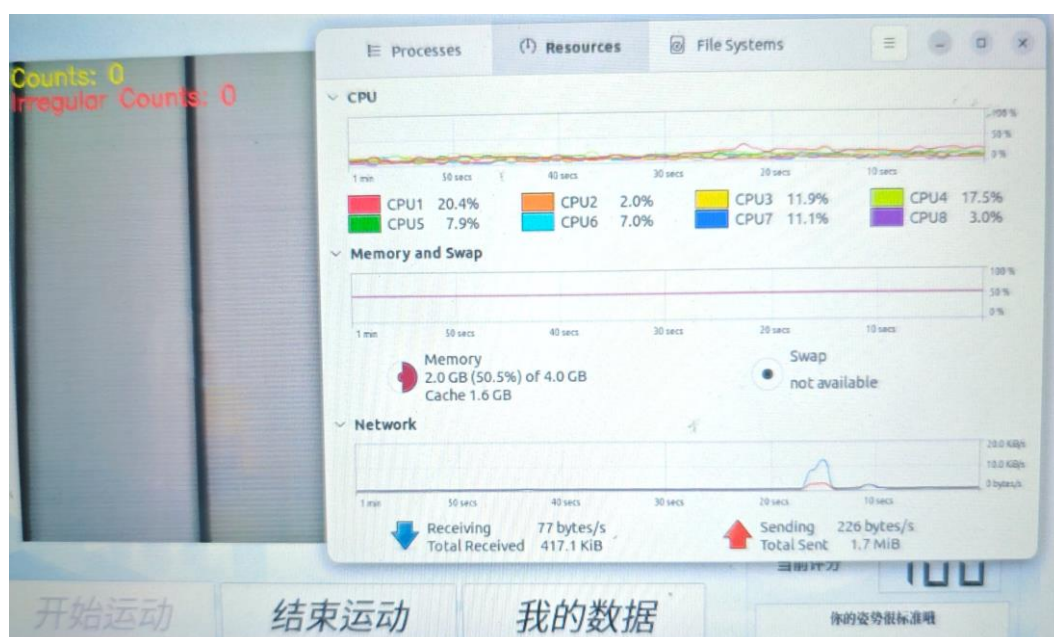


图 30 设备运行情况

第四部分 总结

4.1 可扩展之处

当前系统采用单摄像头方案，在应对极端身体完全遮挡、侧身大角度转体或多人极度混淆的实际场景时，仍可能出现关键点丢失或身份误判。未来改进可从以下三个方向展开：第一，引入多摄像头视角协同或深度相机，实现三维姿态估计，从根本上消除单视角带来的视觉死角；第二，基于系统现有的模块化软件架构，建立动态模型加载机制，快速扩展支持如瑜伽、器械力量训练、球类运动等更丰富的动态动作库；第三，进一步标准化边缘实际场景的测试流程，针对强弱光、突发遮挡、长时运行等复杂边界条件形成固定的性能测试 Benchmark，为系统的工程化落地提供更完备的消融实验与量化数据支撑

4.2 心得体会

在整个智能健身动作分析系统的研发与制作过程中，团队全流程经历了从算法训练、端侧轻量化移植，到软硬件异构联调的完整工程闭环，获得了宝贵的实践经验与深刻的技术体会。

首先，团队深入攻克了深度学习算法在边缘嵌入式平台部署时算力受限的技术瓶颈。在开发初期，直接将未优化的 YOLO 原生模型部署至端侧，导致了极其严重的 CPU 满载过载、设备剧烈发热以及实时运行帧率骤降等问题。面对这一工程困境，我们打破了传统的“纯软件层面优化”思路，查阅芯片底层手册，转向板端硬件级的异构协同调度优化。通过对 ISP 曝光、增益、亮度等参数的调整，实现了复杂光照条件下的正确识别；通过调通底层 librga.so 的 RGA 图形加速引擎与高性能 NPU 推理核心，成功实现了图像采集、尺寸变换、神经网络推理与业务逻辑控制的多硬件流水线并行处理。这使整机视觉推理延迟大幅缩短至 100ms 以内，让我们深刻体会到了“软硬协同”在边缘计算开发中的核心价值。

其次，在算法落地应用中，我们深刻认识到优秀的工程化作品不仅需要实验

室层面的高精度模型，更需要针对复杂光照、局部遮挡等不可控的实际生活场景，设计具备极强抗干扰能力的软件业务状态机。为了解决动作计数在临界切换点由于像素噪点导致的频繁误触发和重计问题，团队反复测试、不断优化 EMA 平滑函数的权重参数，并引入了双阈值阶段转换与最小持续帧校验等多层判定逻辑。当看到系统在弱光、大角度遮挡测试中依然能够准确无误地完成计数与实时纠错反馈时，团队真切体会到了严谨的工程逻辑对于提升系统鲁棒性的关键作用。

最后，本项目的成功落地，极大地提升了团队在边缘计算、多资源协同调度、多终端交互以及数据库应用领域的系统级工程开发能力。这次从零到一的研发历程让我们明白，真正的技术落地必须以解决实际用户痛点为导向，兼顾算法性能、系统功耗与部署成本，这也为我们今后走向更复杂的智慧体育、康复医疗等工业级嵌入式闭环系统开发奠定了坚实的技术基石。

第五部分 参考文献

- [1] Ultralytics. YOLOv8: Ultralytics YOLO object detection and segmentation [EB/OL]. (2023) [2026-02-21]. <https://ultralytics.com/yolov8>.
- [2] JOCHER G, CHAURASIA A, QIU J. Ultralytics YOLOv8 [EB/OL]. (2023) [2026-03-08]. <https://github.com/ultralytics/ultralytics>.
- [3] Google AI Blog. Real-time 3D pose detection with BlazePose [EB/OL]. (2020-08) [2026-04-11]. <https://ai.googleblog.com/2020/08/on-device-real-time-body-pose-tracking.html>.
- [4] scikit-learn developers. Random forest [EB/OL]. [2026-04-13]. <https://scikit-learn.org/stable/modules/ensemble.html#random-forests>.
- [5] Riverbank Computing. PyQt5 documentation [EB/OL]. (2023) [2026-05-07]. <https://www.riverbankcomputing.com/static/Docs/PyQt5/>.
- [6] RONACHER A. Flask documentation (2.3.x) [EB/OL]. (2023) [2026-05-12]. <https://flask.palletsprojects.com/>.
- [7] Google. Android developers [EB/OL]. (2023) [2026-06-19]. <https://developer.android.com/>.
- [8] 齐荣霞. 复杂光照环境下人脸图像预处理优化算法研究[J]. 电脑知识与技术, 2019(33): 218-220.
- [9] 高优, 刘勇, 任力子. 计算机硬件加速技术在图形处理中的应用[J]. 中国信息界, 2026(02): 189-191.